Sentiment Analysis

MLOps Project Report

NLP · Machine Learning · MLflow · DVC · CICD · Flask · AWS

Project Type	Multi-class Text Classification
Domain	Natural Language Processing (NLP)
Dataset	Youtube Comments (37,249 samples)
Best Model	LightGBM (Optuna-Tuned)
MLOps Stack	MLflow + AWS EC2 + S3 + ECR
Date	March 2026

Table of Contents

Table of Contents 2
1. Executive Summary 4
2. Problem Statement 4
  2.1 Business Problem 4
  2.2 Technical Problem Definition 4
  2.3 Challenges 4
3. Dataset Overview 5
  3.1 Data Source 5
  3.2 Dataset Statistics 5
  3.3 Exploratory Data Analysis (EDA) 5
4. Data Preprocessing & NLP Pipeline 6
  4.1 Data Cleaning Steps 6
  4.2 Custom Stop Word Strategy 6
  4.3 Lemmatization with spaCy 6
5. Feature Engineering 6
  5.1 Text Vectorization Approaches Compared 6
  5.2 N-gram Range Selection 7
6. Handling Class Imbalance 7
  6.1 Problem 7
  6.2 Techniques Evaluated (Exp 3) 7
  6.3 Selected Method 8
7. Model Selection & Comparison 8
  7.1 Classifiers Evaluated (Exp 4 — Find Classifier) 8
  7.2 Stacking Ensemble Experiment 8
8. Hyperparameter Optimization with Optuna 8
  8.1 Overview 8
  8.2 Optimization Setup 9
  8.3 Hyperparameter Search Space 9
  8.4 Optimization Result 9
9. MLflow — Experiment Tracking & Model Registry 9
  9.1 MLOps Infrastructure Setup 9
  9.2 MLflow Server Configuration 10
  9.3 Experiment Structure 10
  9.4 MLflow Logged Artifacts Per Run 10
10. Final Model — LightGBM 11

10.1 Why LightGBM? 11
10.2 Final Model Configuration 11
11. Summary of AI & ML Techniques Used 11
12. Pipeline Architecture (DVC) 12
13. Conclusions & Key Findings 13
13.1 Key Findings 13
14. Technology Stack 14
15. Complete MLOps Flow 15
15.1 DVC Pipeline Implementation 15
params.yaml — Centralized Parameter Store 15
15.2 Docker — Containerization 15
15.3 CI/CD Pipeline — GitHub Actions 16
AWS Secrets Management 16
15.4 AWS Infrastructure 17
15.5 End-to-End Deployment Flow Summary 17

1. Executive Summary

This report documents the end-to-end development of a multi-class sentiment classification system trained on youtube comments. The project addresses the problem of automatically identifying whether a Youtube comment expresses a positive, neutral, or negative sentiment. A rigorous, experiment-driven MLOps methodology was adopted throughout, leveraging MLflow for experiment tracking and model registry, AWS EC2 and S3 for scalable infrastructure, and Optuna for automated hyperparameter optimization.

Starting from raw Youtube data with 37,249 comments, the pipeline covers data cleaning, advanced NLP preprocessing, feature engineering with TF-IDF, class imbalance handling, multi-model benchmarking, and hyperparameter tuning. The final production model is a LightGBM classifier achieving a weighted F1-score of approximately 81.3%, selected after systematic comparison across five structured MLflow experiment groups.

2. Problem Statement

2.1 Business Problem

Understanding public sentiment on social media platforms such as Youtube enables a wide range of applications — from brand monitoring and community health tracking to content moderation and trend analysis. Manually labeling thousands of comments is expensive and inconsistent. An automated, scalable sentiment classifier is therefore essential.

2.2 Technical Problem Definition

Given a Youtube comment as plain text, predict its sentiment category from three classes:

- Positive (label 1) — comment expresses a favorable or optimistic opinion
- Neutral (label 0) — comment is factual, balanced, or non-opinionated
- Negative (label -1) — comment expresses criticism, frustration, or opposition

2.3 Challenges

- Class imbalance: Positive (42.5%), Neutral (35.3%), Negative (22.2%) — minority class under-represented
- Noisy text: Youtube comments contain URLs, mentions, emojis, non-ASCII characters, and informal language
- Short and ambiguous texts: Many comments lack sufficient context for reliable classification
- Feature representation: Converting raw text to meaningful numerical features while preserving sentiment signals
- Model selection: Identifying the classifier that best balances accuracy and generalization

3. Dataset Overview

3.1 Data Source

The dataset was sourced from a publicly available Youtube sentiment analysis repository on GitHub. It consists of pre-cleaned Youtube comments paired with sentiment labels.

3.2 Dataset Statistics

Attribute	Value
Total Samples	37,249 comments
Features	clean_comment (text), category (label)
Missing Values	100 null comments, 0 null labels
Duplicate Rows	449 exact duplicates
Final Clean Samples	35,934 after deduplication and null removal
Positive (1)	15,830 samples (42.5%)
Neutral (0)	13,142 samples (35.3%)
Negative (-1)	8,277 samples (22.2%)

3.3 Exploratory Data Analysis (EDA)

The following EDA steps were performed to understand the data distribution and prepare for preprocessing:

- Class distribution visualization via donut chart — revealed moderate imbalance with negative class under-represented
- Comment length distribution by class via KDE plots — showed similar length distributions across all three classes
- Word cloud generation for positive and negative comments — highlighted distinct vocabulary for each sentiment
- Stop-word frequency analysis — identified high-frequency non-informative words for removal
- Character frequency analysis — detected non-ASCII and non-English characters requiring cleaning
- Top N word frequency plots stratified by sentiment — revealed sentiment-specific vocabulary patterns

4. Data Preprocessing & NLP Pipeline

4.1 Data Cleaning Steps

A multi-stage cleaning pipeline was applied to prepare raw text for machine learning:

#	Step	Description
1	Deduplication	Removed 449 duplicate rows to prevent data leakage
2	Null Removal	Dropped 100 rows with missing comment text
3	Lowercasing	Converted all text to lowercase for normalization
4	URL & Mention Removal	Removed http/www URLs and @mentions using regex
5	Non-ASCII Removal	Filtered out non-English characters (e.g. Chinese, Malayalam)
6	Special Character Removal	Stripped punctuation and symbols using regex
7	Number Removal	Eliminated all numeric tokens
8	Lemmatization	Used spaCy (en_core_web_sm) to reduce words to base forms
9	Stop Word Removal	Applied custom expanded stop word list (preserving 'not', 'but', 'no', 'however', 'yet')
10	Short Word Removal	Removed tokens shorter than 3 characters

4.2 Custom Stop Word Strategy

The default NLTK and spaCy stop word lists were augmented with domain-specific terms commonly appearing in Youtube comments (e.g., 'music', 'youtube', 'video'). Critically, negation words such as 'not', 'no', 'but', 'however', and 'yet' were explicitly retained in the vocabulary, as they carry strong sentiment signal and their removal would degrade classifier performance.

4.3 Lemmatization with spaCy

Lemmatization was performed using spaCy's en_core_web_sm model, which reduces inflected word forms to their dictionary root (e.g., 'running' to 'run', 'better' to 'good'). This reduces vocabulary dimensionality and improves generalization compared to simple stemming.

5. Feature Engineering

5.1 Text Vectorization Approaches Compared

Two text-to-vector representations were evaluated via MLflow experiments before selecting the final approach:

Vectorizer	Description	Outcome
Bag of Words (BoW)	CountVectorizer — raw token frequency counts	Lower performance; selected as baseline
TF-IDF	Term Frequency-Inverse Document Frequency weighting	Higher accuracy; selected as final

5.2 N-gram Range Selection

A dedicated MLflow experiment (Exp 2 — TFIDF Trigram max_features) evaluated n-gram ranges from unigrams to trigrams. The trigram range (1,3) was found to capture important multi-word phrases (e.g., 'not very good', 'really love this') that are critical for sentiment understanding. The final configuration used:

- Vectorizer: TF-IDF
- N-gram range: (1, 3) — unigrams, bigrams, and trigrams
- Max features: 10,000 — selected after systematic comparison across 1,000 / 5,000 / 10,000

6. Handling Class Imbalance

6.1 Problem

The dataset exhibited moderate class imbalance: the negative class (22.2%) had roughly half the samples of the positive class (42.5%). Without correction, classifiers tend to be biased toward the majority classes, resulting in poor recall for the minority negative class.

6.2 Techniques Evaluated (Exp 3)

Method	Description
RandomOverSampler	Randomly duplicates minority class samples
SMOTE	Synthetic Minority Over-sampling Technique — creates synthetic minority samples by interpolating between existing ones
ADASYN	Adaptive Synthetic Sampling — generates more samples in harder-to-classify regions
RandomUnderSampler	Randomly removes majority class samples
SMOTEENN	Hybrid: SMOTE oversampling + Edited Nearest Neighbours cleaning
SMOTETomek	Hybrid: SMOTE oversampling + Tomek Links under-sampling
class_weight='balanced'	Adjusts class weights inversely proportional to frequency inside the classifier

6.3 Selected Method

ADASYN (Adaptive Synthetic Sampling) was selected as the final imbalance handling technique. ADASYN generates more synthetic examples in regions of the feature space where the minority class is difficult to classify, leading to better decision boundaries compared to uniform oversampling. All subsequent experiments use ADASYN.

7. Model Selection & Comparison

7.1 Classifiers Evaluated (Exp 4 — Find Classifier)

The following classifiers were benchmarked under identical conditions (TF-IDF trigrams, max_features=10,000, ADASYN balancing) to identify the best-performing architecture:

Classifier	Key Configuration	Notes
Random Forest (Baseline)	n_estimators=100, max_depth=None	Baseline — BoW vectorizer
Random Forest (TF-IDF)	n_estimators=200, max_depth=15	Improved over baseline with TF-IDF
SVM (SVC)	C=0.1, kernel=linear, gamma=scale	Explored but commented out in final code
XGBoost	Default + tuning	Gradient boosting comparison
LightGBM	Optuna-optimized (100 trials)	Best performer — selected
Stacking Ensemble	LightGBM + Logistic Regression (meta-learner)	Marginal improvement, not selected

7.2 Stacking Ensemble Experiment

A stacking classifier was evaluated using LightGBM and Logistic Regression as base estimators, with a Logistic Regression meta-learner. The passthrough=True option was used, meaning the meta-learner received both the base model predictions and the original TF-IDF features. Despite this, the stacking ensemble did not produce a meaningful accuracy improvement over standalone LightGBM, so the simpler LightGBM model was retained for production.

8. Hyperparameter Optimization with Optuna

8.1 Overview

Optuna is an automatic hyperparameter optimization framework that uses Bayesian-style search strategies (Tree-structured Parzen Estimator — TPE) to efficiently explore the hyperparameter space, avoiding the combinatorial explosion of grid search.

8.2 Optimization Setup

- Framework: Optuna v4.7.0
- Search strategy: TPE (Tree-structured Parzen Estimator)
- Number of trials: 100
- Optimization metric: Weighted F1-score
- Model: LightGBM multiclass classifier

8.3 Hyperparameter Search Space

Hyperparameter	Best Value Found	Description
n_estimators	886	Number of boosting rounds
learning_rate	0.0628	Step size for gradient update
max_depth	15	Maximum tree depth
num_leaves	47	Number of leaves per tree
min_child_samples	14	Minimum samples in a leaf node
colsample_bytree	0.780	Fraction of features used per tree
subsample	0.501	Fraction of samples used per iteration
reg_alpha (L1)	0.277	L1 regularization — sparsity
reg_lambda (L2)	1.045	L2 regularization — weight decay

8.4 Optimization Result

After 100 trials, the best LightGBM model achieved a weighted F1-score of 0.8127 (81.27%) on the validation set. The Optuna study converged progressively, with Trial 97 being the final best-performing configuration.

9. MLflow — Experiment Tracking & Model Registry

9.1 MLOps Infrastructure Setup

A production-grade MLflow tracking server was deployed on AWS to enable persistent experiment logging, artifact storage, and model versioning. The infrastructure setup involved:

- AWS IAM user creation with AdministratorAccess for programmatic access
- AWS CLI configuration with access keys, region (us-east-1), and output format
- Amazon S3 bucket creation (mlflow-bckt-1) for artifact storage
- AWS EC2 Ubuntu instance launch with port 5000 opened in security groups
- MLflow server installation on EC2 via pipenv with boto3 and awscli dependencies
- MLflow server launched with SQLite backend store and S3 artifact root

9.2 MLflow Server Configuration

The MLflow tracking server was started on the EC2 instance with the following configuration, making it accessible at http://ec2-<public-ip>.compute-1.amazonaws.com:5000/:

- Backend store: SQLite (mlflow.db) — stores parameters, metrics, and run metadata
- Default artifact root: s3://mlflow-bckt-1 — stores model files, plots, and serialized objects
- Host: 0.0.0.0 / Port: 5000 — publicly accessible
- CORS: Allowed from all origins for notebook access

9.3 Experiment Structure

Five structured MLflow experiments were created, each addressing a distinct research question in the model development lifecycle:

#	Experiment Name	Purpose
1	Random Forest Baseline	Compare BoW vs TF-IDF vectorization with Random Forest
2	Exp 2 — TFIDF Trigram max_features	Find optimal max_features (1K, 5K, 10K, 20K) with trigrams
3	Exp 3 — Handling imbalanced_data	Compare 7 class imbalance handling techniques
4	Exp 4 — Find Classifier	Benchmark SVM, RF, XGBoost, LightGBM classifiers
5	Stacking Experiment	Evaluate LightGBM + LR stacking ensemble

9.4 MLflow Logged Artifacts Per Run

For every experiment run, the following information was systematically tracked in MLflow:

- Parameters: vectorizer_type, ngram_range, max_features, n_estimators, max_depth, learning_rate, imbalance_method, classifier name
- Metrics: Accuracy, Precision (weighted), Recall (weighted), F1-score (weighted), ROC-AUC score
- Tags: Run name, experiment type, model type, description
- Artifacts: Confusion matrix heatmap (PNG), serialized model file, classification report

10. Final Model — LightGBM

10.1 Why LightGBM?

LightGBM (Light Gradient Boosting Machine) was selected as the final production model based on several advantages that make it well-suited for this task:

- Leaf-wise tree growth: Unlike depth-wise growers (e.g., Random Forest, XGBoost), LightGBM grows the leaf with the most loss reduction at each step, leading to better accuracy with fewer trees
- Efficient sparse matrix handling: Particularly beneficial for high-dimensional TF-IDF feature spaces
- Built-in L1 and L2 regularization: Controls overfitting via reg_alpha and reg_lambda
- Native multi-class support: Uses the softmax (multiclass) objective directly
- Speed: Histogram-based algorithm significantly reduces memory usage and training time

10.2 Final Model Configuration

Component	Configuration
Vectorizer	TF-IDF, ngram_range=(1,3), max_features=10,000
Imbalance Handling	ADASYN (Adaptive Synthetic Sampling)
Classifier	LightGBM (LGBMClassifier)
Objective	multiclass (softmax), num_class=3
n_estimators	886
learning_rate	0.06281921270935097
max_depth	15
num_leaves	47
L1 regularization	0.277 (reg_alpha)
L2 regularization	1.045 (reg_lambda)
Best Weighted F1-Score	~81.3%

11. Summary of AI & ML Techniques Used

Category	Technique / Tool	Purpose
NLP Preprocessing	spaCy (en_core_web_sm)	Lemmatization and linguistic processing
NLP Preprocessing	NLTK	Stop word lists and tokenization

Category	Technique / Tool	Purpose
NLP Preprocessing	Regex (re module)	URL, mention, and character cleaning
Feature Engineering	TF-IDF (scikit-learn)	Text vectorization with n-grams (1,3)
Feature Engineering	Bag of Words (BoW)	Baseline vectorization (CountVectorizer)
Class Imbalance	ADASYN	Adaptive synthetic oversampling (selected)
Class Imbalance	SMOTE, SMOTETomek, SMOTEENN	Alternative oversampling techniques evaluated
Class Imbalance	RandomOverSampler / UnderSampler	Simple resampling baselines evaluated
Classification	LightGBM (final)	Gradient boosting — best classifier
Classification	Random Forest	Baseline and intermediate experiments
Classification	XGBoost	Alternative gradient boosting evaluated
Classification	SVM (SVC)	Explored for linear separability
Ensemble Learning	Stacking Classifier	LightGBM + LR meta-learner (evaluated, not deployed)
Hyperparameter Tuning	Optuna (TPE)	100-trial Bayesian hyperparameter search
Evaluation	Accuracy, F1, Precision, Recall, ROC-AUC	Multi-metric performance tracking
MLOps Tracking	MLflow	Experiment tracking, parameter/metric logging, model registry
Cloud Infrastructure	AWS EC2 + S3	MLflow server hosting and artifact storage
Cross-Validation	StratifiedKFold (k=5)	Used in stacking ensemble to prevent class-imbalance bias

12. Pipeline Architecture (DVC)

The project concludes with a plan to formalize the full ML pipeline using DVC (Data Version Control), enabling reproducible, versioned pipeline stages. The proposed five-stage DVC pipeline is:

Stage	Name	Description
1	Data Ingestion	Load raw Youtube CSV from source and save locally
2	Data Preprocessing	Text cleaning, lemmatization, stop word removal, deduplication
3	Model Building	TF-IDF vectorization, ADASYN balancing, LightGBM training

Stage	Name	Description
4	Model Evaluation with MLflow	Log metrics, parameters, confusion matrix, and artifacts to MLflow
5	Model Registration with MLflow	Register best model to MLflow Model Registry for deployment

13. Conclusions & Key Findings

13.1 Key Findings

- TF-IDF with trigrams significantly outperformed Bag of Words for this task, capturing multi-word sentiment expressions
- ADASYN was the most effective technique for handling class imbalance, improving recall on the minority negative class
- LightGBM consistently outperformed Random Forest, XGBoost, and SVM in all comparable experiments
- Optuna's Bayesian search converged to a strong configuration (F1 ~81.3%) within 100 trials
- Stacking ensemble did not yield meaningful gains over a well-tuned single model
- MLflow on AWS provided reliable, scalable experiment tracking with automatic artifact versioning in S3

14. Technology Stack

Category	Library / Tool	Version
ML Framework	scikit-learn	Latest
Gradient Boosting	LightGBM	4.6.0
Gradient Boosting	XGBoost	Latest
NLP	spaCy	en_core_web_sm
NLP	NLTK	Latest
Imbalanced Learning	imbalanced-learn	Latest (SMOTE, ADASYN, etc.)
HPO	Optuna	4.7.0
MLOps	MLflow	3.10.0
Cloud Storage	AWS S3 (boto3)	1.42.59
Cloud Compute	AWS EC2 (Ubuntu)	us-east-1
Data Processing	pandas, numpy	Latest
Visualization	matplotlib, seaborn, wordcloud	Latest
Pipeline (Planned)	DVC	Planned

15. Complete MLOps Flow

The full MLOps lifecycle was implemented end-to-end: from data versioning and reproducible pipeline execution with DVC, through containerized model serving with Docker, to automated build and deployment via a GitHub Actions CI/CD pipeline targeting AWS ECR and EC2.

Complete flow: Code Push → GitHub Actions (CI/CD) → Docker Build → AWS ECR → AWS EC2 (self-hosted runner) → Flask App (port 8080)

15.1 DVC Pipeline Implementation

DVC (Data Version Control) was used to define a fully reproducible, parameterized machine learning pipeline in a dvc.yaml file. Each stage declares its command, dependencies, parameters, and outputs, enabling DVC to detect what has changed and re-run only the necessary stages. The pipeline connects seamlessly to MLflow for evaluation tracking and model registration.

#	Stage	Command	Key Inputs / Params	Outputs
1	data_ingestion	python src/data_ingestion.py	src/data_ingestion.py	data/raw/
2	data_preprocessing	python src/data_preprocessing.py	params.yaml: preprocessing.test_size; dep: data/raw/data.csv	data/processed/
3	model_building	python src/model_building.py	14 params from params.yaml (vectorization + LightGBM hyperparams); deps: train.csv, test.csv	model/lgbm_model.pkl, model/tfidf_vectorizer.pkl
4	model_evaluation	python src/model_evaluation.py	deps: test.csv, lgbm_model.pkl, tfidf_vectorizer.pkl	experiment_info.json (MLflow run metadata)
5	register_model	python src/register_model.py	deps: register_model.py, experiment_info.json	Model registered in MLflow Model Registry

params.yaml — Centralized Parameter Store

All configurable values are stored in a single params.yaml file that DVC reads at runtime. This ensures consistency across pipeline runs and makes hyperparameter changes fully traceable. The model_building stage reads 14 parameters covering vectorization settings and all LightGBM hyperparameters found by Optuna.

15.2 Docker — Containerization

The Flask inference application is packaged in a Docker container to guarantee environment consistency between development and production. The image is based on python:3.11-slim-bookworm for a minimal footprint, with only the necessary system libraries for LightGBM (libgomp1, build-essential) and the application's Python dependencies installed from requirements.txt.

- Base image: python:3.11-slim-bookworm — minimal Debian-based image keeps container size small
- System dependency: libgomp1 installed via apt-get — required by LightGBM for OpenMP multi-threading support
- Working directory: /app — application code and model artifacts copied into the container
- Exposed port: 8080 — Flask app listens on this port, mapped to host port 8080
- Entrypoint: python3 flask_app/app.py — Flask inference server started automatically on container launch
- --restart always flag applied at runtime — ensures container auto-restarts on EC2 reboot or crash
- AWS credentials injected as environment variables at runtime — no secrets baked into the image

15.3 CI/CD Pipeline — GitHub Actions

A three-job GitHub Actions workflow (workflow.yaml) automates the full integration, delivery, and deployment cycle. It is triggered on every push to the main branch (excluding README changes). AWS credentials are stored as GitHub Secrets and never exposed in the workflow file.

Job	Runner	Steps Performed
1. integration (CI)	ubuntu-latest (GitHub)	Checkout code → Lint code → Run unit tests
2. build-and-push-ecr-image (CD)	ubuntu-latest (GitHub)	Checkout → Configure AWS credentials (Secrets) → Login to Amazon ECR → docker build → docker push to ECR (tag: latest)
3. Continuous-Deployment	self-hosted (AWS EC2)	Login to ECR → docker pull latest image → Stop and remove old container → docker run (port 8080, --restart always, AWS env vars) → docker system prune

AWS Secrets Management

All sensitive values are stored as GitHub Secrets and referenced using the ${{ secrets.* }} syntax. The five required secrets are:

- AWS_ACCESS_KEY_ID — IAM programmatic access key
- AWS_SECRET_ACCESS_KEY — IAM secret for authentication
- AWS_REGION — target deployment region

- ECR_REPOSITORY_NAME — name of the ECR repository holding the Docker image
- AWS_ECR_LOGIN_URI — full ECR registry URI used to pull and run the image on EC2

15.4 AWS Infrastructure

Three AWS services form the production infrastructure backbone:

Service	Role	Details
Amazon S3	Artifact storage	Bucket mlflow-bckt-1 stores all MLflow artifacts: model files, confusion matrix plots, vectorizer pickles. Accessible by both the MLflow server and EC2 inference instance via IAM credentials.
Amazon ECR	Container registry	Elastic Container Registry stores the Docker image tagged as latest. GitHub Actions pushes to ECR on every successful CI run; the EC2 runner pulls the image directly from ECR for deployment.
Amazon EC2 (Ubuntu)	App server + self-hosted runner	Hosts the Flask application container (port 8080) and acts as the GitHub Actions self-hosted runner for the Continuous-Deployment job. Port 8080 opened in EC2 security group for public inference access.

15.5 End-to-End Deployment Flow Summary

The following numbered sequence describes the complete journey from a code change to a live prediction API:

- **Developer pushes to main** — triggers the GitHub Actions workflow automatically
- **CI job runs** — code linting and unit tests pass on a GitHub-hosted ubuntu runner
- **CD build job runs** — Docker image built from the repository's Dockerfile and pushed to Amazon ECR tagged as "latest"
- **Deployment job runs on EC2 self-hosted runner** — pulls the new image from ECR, stops the old container gracefully, and launches the new container with --restart always
- **Flask API is live** — sentiment predictions available at http://[EC2-Public-IP]:8080/ with zero-downtime rolling update
- **Cleanup** — docker system prune removes dangling images and containers to keep disk space free on EC2